This has two important results. First, it guarantees consistency: the same policy logic applies to both direct access and AI-mediated access. Second, it structurally enforces safety. Unauthorized data is never retrieved and therefore never enters the model's context window.

What the notebook shows

The Jupyter notebook walks through the following steps:

- Evaluating authorization policies using typed partial evaluation rather than a binary decision
- Inspecting the resulting policy residual to understand the authorized scope
- Translating that residual into a retrieval constraint

In a production system, this constraint could be used to search a vector database; however, in this demonstration, we simply translate the residual into the filter.

Why this matters

This example demonstrates the main point of this section: authorization-before-retrieval is not just a quick fix or a recommended practice. Instead, it is a design pattern that prevents certain types of failures. By integrating access control into the retrieval process and basing it on policy evaluation, systems can securely combine AI-driven interactions with sensitive enterprise data.

Readers interested in exploring the mechanics more deeply can use the notebook to experiment with different authorization scenarios and observe how policy changes influence retrieval behavior. The notebook is meant to be exploratory rather than prescriptive, emphasizing that enforcement remains deterministic while AI functions as an assistant rather than an authority.